

trust graduation

Trust Graduation Protocol

GitHub, npm, protocol, schema, and roadmap documentation packet.

Trust Graduation

Every agent product is rebuilding the permission layer. Trust Graduation is the embeddable standard for human-gated agent autonomy: earned per action class, on real evidence, with a universal approval payload. Embed in six lines. Apache-licensed. Zero dependencies. Reference implementation: Mission by Phenomena Labs Ltd.

Most agent systems ask: what can this agent automate?

Trust Graduation asks: what has this agent earned the right to do for this user, in this action class, under these constraints?

The long-form thinking is in MANIFESTO.md.

Install

```
npm install @trust-graduation/core
```

Six-line Embed

```
import { TrustGraduation } from "@trust-graduation/core"; / const tg = new TrustGraduation({ workspace: "user-123",  
evidence: localLedger }); / const decision = tg.canExecute({ actionClass: "email.send.external", context: {  
recipient, body } }); / if (decision.allowed) await actuallySend(); / else if (decision.needsApproval) await  
pushApprovalToUser(decision.packet);
```

External sends, public posts, money movement, legal commitments, and policy changes stay approval-gated by default.

Embedded By

Early adopters will be listed here.

Coming Next

Trust Graduation is intended as a small protocol family, not a single SDK. The next primitives stay gated on adoption: do not ship a new package until the prior primitive has at least one external embed or partner review.

- @trust-graduation/receipts — universal evidence that an agent-mediated action happened. Preview: schemas/v2/receipts.schema.json.
- @trust-graduation/open-loops — shared pending-work shape so agents can claim, close, dedupe, and hand off work.
- @trust-graduation/voice — portable user voice profile and draft-conformance payloads.
- @trust-graduation/federation — optional consented hosted layer for cross-product evidence, receipts, loops, and voice.

This week: autonomy. Next: receipts. Then: open loops. Then: voice. All under one open standard.

What This Repo Contains

- `src/` — zero-dependency JavaScript reference implementation.
- `schemas/v1/` — JSON schemas for action classes, evidence, decisions, approval packets, and license entitlements.
- `schemas/v2/receipts.schema.json` — forward-design preview for the receipts primitive; not a shipped package yet.
- `docs/spec-overview.md` — one-page spec overview.
- `docs/spec-deep-dive.md` — compact v1 protocol details.
- `docs/receipts-forward-design.md` — rationale and storage-agnostic API sketch for future receipts work.
- `docs/github-npm-publishing.md` — what belongs in GitHub and npm, plus release checklist.
- `docs/pdf/trust-graduation-protocol.pdf` — printable protocol packet generated with `npm run docs:pdf`.
- `examples/minimal.js` — minimal embed example.
- `packages/python/` and `packages/go/` — package placeholders for the next language ports.

Documentation

Generate the printable protocol packet:

```
npm run docs:pdf
```

The PDF is written to `docs/pdf/trust-graduation-protocol.pdf`. It is included in the npm package because `package.json` includes `docs/` in `files`.

Core Concepts

- Action class: the specific kind of thing an agent wants to do, such as `draft.response` or `email.send.external`.
- Evidence ledger: approvals, edits, rejections, receipts, outcomes, trust issues, and rollbacks.
- Autonomy level: the current earned capability for an action class.
- Approval packet: a standard payload any agent product can render when a human decision is required.
- License entitlement: a free-stage token payload for protocol features such as `core`, `schemas`, `approval-packets`, `local-evidence`, and future federation.

License Entitlements

The package defaults to a free local protocol license. Tokens are intentionally simple in alpha: `tg1.<base64url-json>`. They do not contact a server and they do not change the Trust Graduation decision contract.

```
import { createLicenseToken, decodeLicenseToken, licenseAllows } from "@trust-graduation/core"; / const token =
createLicenseToken({ subject: "workspace-123", features: ["core", "schemas"] }); / const status =
decodeLicenseToken(token); / if (licenseAllows(status, "core")) { / // run the local reference implementation / }
```

Future hosted federation or enterprise support can issue stronger tokens without changing how an embedder calls `canExecute()`.

Status

Package status: 0.1.0-alpha.2.

Schema status: draft schemas/v1/.

The schemas describe the protocol shape. The package is the reference implementation. Schemas reach stable v1.0 after three external implementations or integrations, one public adopter, and no breaking schema changes for 30 days. Package 1.0.0 follows when the JavaScript API matches stable schemas and has CI-covered compatibility tests.

License

Apache-2.0.

Trust Graduation 1.0 Overview

Status: alpha draft

Trust Graduation is a standard for permissioned AI agent autonomy.

The protocol does not grant global trust. It evaluates one action class at a time, using evidence from a user's actual approvals, edits, rejections, receipts, outcomes, trust issues, and rollbacks.

Core Rule

An agent may execute only what it has earned for this user, in this action class, under these constraints.

Autonomy Ladder

Level	Name	Meaning
-------	------	---------

---	---	---
-----	-----	-----

0	Observe	Read, summarize, flag, and explain.
---	---------	-------------------------------------

1	Prepare	Draft, rank, route, and assemble review packets.
---	---------	--

2	Stage	Pre-fill local packets or commands for final review.
---	-------	--

3	Execute Narrow	Execute bounded low-risk actions with receipts.
---	----------------	---

4	Delegate	Run bounded workflows inside a defined domain.
---	----------	--

5	Govern	Recommend policy or autonomy changes.
---	--------	---------------------------------------

Default Boundary

External sends, public posts, calendar invites, money movement, legal commitments, irreversible changes, and permission changes require human approval unless a separate policy explicitly says otherwise.

Entitlement Boundary

The reference package includes a free-stage license entitlement payload so protocol features can be gated later without changing the decision contract. Entitlements govern access to protocol capabilities such as core, schemas, approval-packets, local-evidence, and future federation. They do not grant autonomy. Only evidence plus policy can do that.

Public Promise

Never sends without approval.

Trust Graduation 1.0 Deep Dive

Status: alpha draft

Artifacts

Trust Graduation 1.0 defines four core autonomy payloads:

- Action class policy
- Evidence event
- Decision
- Approval packet

Schemas live in `schemas/v1/`.

The reference implementation also defines a license entitlement payload. It is not an autonomy payload and cannot make an action executable; it only gates product/protocol capabilities such as package features, schema access, approval-packet rendering, local evidence helpers, and future federation.

Evidence Tiers

The reference implementation maps evidence into four tiers:

| Tier | Rule |

|---|---|

| gated | Default until evidence is strong enough. |

| supervised | At least 5 positive events, 3 approvals, no negative evidence, low edit distance. |

| auto_capped | At least 10 positive events, 6 approvals, no negative evidence, very low edit distance. |

| review | Any trust issue, repeated rejections, rollback, or high rejection rate. |

Decision Contract

`canExecute()` returns:

- `allowed`: whether the agent can execute now
- `needsApproval`: whether a human approval packet is required
- `mode`: current handling mode
- `autonomyLevel`: earned level from 0-5
- `tier`: evidence tier
- `policy`: action class policy

- evidence: evidence summary
- packet: standard approval payload when needed

Federation Boundary

Federated evidence is not implemented in this alpha package. The package is designed so an optional hosted federation layer can provide the same evidence array from multiple products with user consent.

License Entitlement Contract

Alpha entitlement tokens use the prefix `tg1` and encode a JSON payload matching `schemas/v1/license-token.schema.json`.

Required fields:

- version
- issuer
- subject
- product
- plan
- features
- modules

Default free local protocol features are `core`, `schemas`, `approval-packets`, and `local-evidence`. The entitlement layer is deliberately separate from `canExecute()` so monetization cannot accidentally bypass Trust Graduation.

Receipts Forward Design

Trust Graduation core decides what an agent has earned the right to do. Receipts are the evidence that something actually happened.

This document is forward design, not a shipped package. `@trust-graduation/receipts` should only ship after `@trust-graduation/core` has at least one external embed.

Why Receipts Exist

Every agent product needs an audit trail. Without a shared receipt shape, each agent keeps private logs that other agents cannot use. A Trust Graduation receipt makes completed work legible across products:

- Core can treat receipts as evidence for autonomy graduation.
- Other agents can avoid duplicating work that has already closed.
- Users get one audit trail for agent-mediated actions.
- Future federation can share evidence across products with explicit consent.

Minimal Shape

```
{ / "protocol": "trust-graduation-receipts", / "version": "1.0", / "receiptId": "rcpt_20260531_abc", / "workspace":
"user-ronen", / "agent": "cursor", / "actionClass": "code.commit", / "result": "completed", / "evidence": { /
"target": "commit:abc123", / "summary": "Added auth middleware", / "approvedBy": "human:ronen", / "approvedAt":
"2026-05-31T10:00:00Z", / "completedAt": "2026-05-31T10:00:30Z", / "humanEditDistance": 0.05, / "outcome": "merged"
/ }, / "links": { / "approvalPacketId": "tg_xyz", / "openLoopId": "ol_abc", / "ruleEvidence":
["rule:proof-backed-followups"] / } / }
```

Package Sketch

```
import { ReceiptStore } from "@trust-graduation/receipts"; / const store = new ReceiptStore({ workspace:
"user-ronen" }); / const receipt = store.append({ / agent: "my-agent", / actionClass: "email.send.external", /
result: "completed", / evidence: { / target: "msg:xyz", / summary: "Sent approved follow-up" / } / }); / const
recent = store.query({ / actionClass: "email.send.external", / since: "2026-05-24T00:00:00Z", / federated: true /
});
```

The package should stay storage-agnostic: local file, product database, and `trust-graduation.org/api` federation can all satisfy the same interface.

Discipline Gate

Do not ship `@trust-graduation/receipts` until:

- `@trust-graduation/core` has one external embed.
- Mission emits receipts internally in a way that maps cleanly to the schema.
- One external partner wants receipt emission or consumption enough to review the schema.

Until then, the only artifact is the schema preview at `schemas/v2/receipts.schema.json`.

Protocol Adoption Roadmap

Status: alpha operating plan

Phase 1: Crystallize

Goal: a public spec and package any agent vendor can embed in less than 100 lines.

- Keep the reference implementation pure: JSON in, JSON out.
- Publish the npm package first.
- Keep Python and Go ports API-compatible with the JavaScript decision contract.
- Publish schemas under /schemas/v1/.
- Keep license entitlement tokens free-local by default; use them to prove the future monetization boundary without blocking adoption.
- Use the Manifesto as launch positioning, not as a substitute for an embeddable package.

Phase 2: Seed Adoption

Goal: three named external integrations, including one credible agent product or framework.

Pitch:

> We will help you embed Trust Graduation in one day. You get an approval/evidence safety layer and a standard audit trail. We get the citation.

Good first targets:

- agent frameworks with active maintainers
- open-source products where a PR can be reviewed publicly
- AI infra teams already wrestling with autonomy gates

Phase 3: Federation

Goal: opt-in cross-product evidence sharing.

The alpha package does not implement federation. It is shaped so a hosted service can later provide the same evidence ledger from multiple products with user consent.

Phase 4: Governance

Goal: move from a Mission-originated reference implementation to a citeable standard.

Governance should remain lightweight until there are real adopters. Do not create process before adoption.

Phase 5: Monetization

Potential protocol-side revenue:

- hosted federation
- compliance/audit pack
- enterprise support contracts
- white-label implementation support

The first monetization gate is not pricing. It is external adoption.

In alpha, entitlement tokens should stay permissive. The purpose is to make future hosted federation, compliance packs, and enterprise support attach cleanly to protocol features after adoption, not to add friction before the standard has citations.

GitHub And npm Publishing Guide

Trust Graduation public artifacts should make the protocol easy to inspect, install, and cite.

What Goes To GitHub

The GitHub repo is the public protocol home. It should include:

- README.md - hook, install, six-line embed, status, roadmap, and links.
- src/ - zero-dependency JavaScript reference implementation for @trust-graduation/core.
- schemas/v1/ - stable alpha schemas for core autonomy primitives.
- schemas/v2/receipts.schema.json - forward-design preview for receipts, not a shipped package.
- docs/spec-overview.md - one-page spec overview.
- docs/spec-deep-dive.md - compact protocol details.
- docs/receipts-forward-design.md - next-pillar design note.
- docs/adoption-roadmap.md - external embed path and discipline gates.
- docs/github-npm-publishing.md - this publishing map.
- docs/pdf/trust-graduation-protocol.pdf - printable protocol packet.
- examples/minimal.js - six-line embed example.
- packages/python/README.md and packages/go/README.md - language-port placeholders until parity work begins.
- CONTRIBUTING.md, CHANGELOG.md, LICENSE.

The repo should not include private Mission workspace data, customer context, local receipts, secrets, unpublished GTM notes, or product-specific Mission Control internals.

What Goes To npm

The npm package is the embeddable runtime. It should include:

- src/ - runtime implementation and TypeScript declarations.
- schemas/ - protocol schemas.
- docs/ - public docs and PDF protocol packet.
- examples/ - minimal embed examples.
- packages/*/README.md - language-port status.
- README.md, LICENSE.

The npm package should not include:

- local workspace data
- .git
- private strategy notes

- generated logs
- secrets or connector credentials
- unpublished partner/customer context

The current package boundary is controlled by `package.json` files.

Release Checklist

Before publishing:

- Run `npm test`.
- Run `npm run docs:pdf`.
- Verify `README.md` install and import snippets use `@trust-graduation/core`.
- Verify `docs/pdf/trust-graduation-protocol.pdf` exists and renders.
- Verify `package.json` version and `CHANGELOG.md` agree.
- Run `npm pack --dry-run` and inspect included files.
- Publish only after the GitHub repo is pushed.

Discipline Gate

Do not ship `@trust-graduation/receipts`, `@trust-graduation/open-loops`, `@trust-graduation/voice`, or federation packages until the prior primitive has external embed traction. Forward-design schemas are allowed; runtime packages wait for demand.